

一. (20 points) NLP 基础

1. RNN 和 Transformer 在处理序列数据时的主要机制是什么? (3 points)
2. 为什么 Transformer 在处理长序列时通常优于传统 RNN (如 LSTM/GRU)? 给出具体解释。 (3 points)
3. 从计算效率角度, 分析 Transformer 相对于 RNN 的优势和劣势。 (4 points)
4. 针对 RNN 和 Transformer, 各举一个更适合的应用场景, 并说明原因。 (4 points)
5. 单塔模型和双塔模型各自的优缺点是什么? (3 points)
6. BLEU 作为度量指标有哪些缺陷? (3 points)

1.1

RNN 通过循环结构逐个处理序列元素, 维护隐藏状态传递历史信息, 当前输出依赖前一时刻的隐状态

Transformer 基于自注意力机制, 并行处理所有序列位置的 token, 通过注意力权重动态捕捉全局依赖关系

1.2

RNN (如 LSTM/GRU) 因序列逐步传递存在梯度消失/爆炸问题, 难以建模长距离依赖

Transformer 的自注意力机制允许任意位置直接交互, 路径长度为常数, 且多头注意力能捕获多维度依赖关系, 并行计算进一步提升了长序列处理效率

1.3

优势: 训练时序列可并行计算 (时间复杂度为 $O(n^2 \cdot d)$, 但并行度高), 远快于 RNN 的 $O(n \cdot d^2)$ 串行计算

劣势: 推理时自注意力复杂度为 $O(n^2)$, 内存消耗随序列长度平方增长, 而 RNN 的推理复杂度仅为 $O(n \cdot d^2)$, 更适合超长序列流式处理

1.4

RNN: 实时语音识别 (需逐帧处理输入, 低延迟场景适合 RNN 的流式特性)

Transformer: 机器翻译 (需全局上下文对齐, 自注意力能有效建模跨语言长距离依赖)

1.5

单塔: 优点为交互充分 (如 BERT 用于句子对任务), 缺点为计算代价高 (需组合所有输入对)

双塔: 优点为高效检索 (独立编码后计算相似度, 适合大规模匹配), 缺点为表征交互不足 (如可能丢失细粒度对齐信息)

1.6

缺陷:

- 仅基于 n-gram 表面匹配, 忽略语义相似性 (如同义词替换得零分)
- 对句子流畅度、语法正确性无判断
- 短文本评价不合理 (易因 brevity penalty 惩罚合理输出)
- 依赖领域适配的参考译文, 多样性不足时评分偏差大

二. (35 points) Transformer

1. 一层 Transformer 中都有哪些模块包含参数? (2 points)

2. 给定如下 Transformer, 给定维度 $d_{model} = 512$, FFN 中间层维度 $d_{ff} = 2048$, 头数 $h = 8$, 头维 $d_q = d_k = d_v = 64$, 求 Transformer 一层的参数量有多少。 (3 points)

3. 通常, 我们可以将 Transformer 的 FFN 表示成如下形式:

$$FF(\mathbf{x}) = f(\mathbf{x} \cdot W_1 + b_1) \cdot W_2 + b_2$$

在 Memory Network(MN) 中, 一个 neural memory 中有 d_m 个 key-value pair, 我们称之为 memory, 每一个键我们用 $k_i \in \mathbb{R}^d$ 表示, 总共我们可以得到一个参数矩阵 $K \in \mathbb{R}^{d_m \times d}$ 。同理, 我们可以得到值的参数矩阵 $V \in \mathbb{R}^{d_m \times d}$ 。给定一个输入向量 $\mathbf{x} \in \mathbb{R}^d$, 我们可以计算 key 的分布, 然后计算输出:

$$p(k_i | x) \propto \exp(\mathbf{x} \cdot \mathbf{k}_i)$$

$$MN(\mathbf{x}) = \sum_{i=1}^{d_m} p(k_i | \mathbf{x}) \mathbf{v}_i$$

上述过程, 写成矩阵的形式为:

$$MN(\mathbf{x}) = \text{softmax}(\mathbf{x} \cdot \mathbf{K}^\top) \cdot V$$

因此, 忽略 FFN 中的偏置后, 我们可以从另外一个角度看待 FFN。Transformer 中的 FFN 可以看做是一种储存了知识的 memory network。请推导出他们二者之间的联系。 (10 points)

4. Transformer 架构具有多头注意力机制。他们将其描述为一种拼接过程, 用于在每一层中并行地为多个“头” ($\text{head}_j \in \mathbb{R}^d, j \leq J$) 执行注意力函数。核心公式如下:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where } \text{head}_j = \text{Attention}(QW_j^Q, KW_j^K, VW_j^V)$$

实际上, 我们也可以将多头注意力的结果看成是多个头的结果的累加, 即每个 head_j 与矩阵中一部分 W^O 乘积并累加的结果。请你尝试推导。[提示: 从分块矩阵的角度出发] (10 points)

2.1

1. 多头自注意力模块:

(a) 每个注意力头都有三个权重矩阵: 查询 (Query) 权重矩阵 W_Q 、键 (Key) 权重矩阵 W_K 和值 (Value) 权重矩阵 W_V , 以及偏置

(b) 投影矩阵 W^O , 包含权重和偏置

2. 前馈神经网络 (FFN) 模块: 两个线性变换矩阵 W_1, W_2 及其偏置项 b_1, b_2 (若包含偏置)

3. Layer Norm 层: 缩放因子 γ 、偏移因子 β

2.2

MHA 部分:

每个头的 $Q/K/V$ 权重矩阵参数量: $3 \times d_{\text{model}} \times d_{\text{head}} = 3 \times 512 \times 64 = 98304$

$h = 8$ 个头的总参数量: $8 \times 98,304 = 786432$, 对应偏置为 $3 \times 512 = 1536$

输出投影矩阵 W^O 参数量: $h \cdot d_v \times d_{\text{model}} = 8 \times 64 \times 512 = 262144$, 对应偏置为 512

MHA 总参数量: $786432 + 262144 + 1536 + 512 = 1050624$

FFN 部分:

第一层参数量: $d_{\text{model}} \times d_{\text{ff}} = 512 \times 2048 = 1048576$, 对应偏置为 2048

第二层参数量: $d_{\text{ff}} \times d_{\text{model}} = 2048 \times 512 = 1048576$, 对应偏置为 512

FFN 总参数量: $1048576 + 1048576 + 2048 + 512 = 2099712$

Layer Norm 部分: 2 个 Layer Norm 层总参数量: $2 \times 2 \times d_{\text{model}} = 2048$

所以一层总参数量: $1050624 + 2099712 + 2048 = 3152384$

2.3

Transformer 中的 FFN 可以表示为:

$$FF(\mathbf{x}) = f(\mathbf{x} \cdot W_1 + b_1) \cdot W_2 + b_2$$

Memory Network 的记忆网络表示为:

$$MN(\mathbf{x}) = \sum_{i=1}^{d_m} p(k_i | \mathbf{x}) \mathbf{v}_i$$

其中, $p(k_i | \mathbf{x}) \propto \exp(\mathbf{x} \cdot \mathbf{k}_i)$, 并且可以写成矩阵形式:

$$MN(\mathbf{x}) = \text{softmax}(\mathbf{x} \cdot K^T) \cdot V$$

其中, K 和 V 分别为键和值矩阵, $\mathbf{x}$ 为输入向量。

观察到 Transformer 中的 FFN 和 Memory Network 具有相似的结构。FFN 中的输入 $\mathbf{x}$ 会与 W_1 相乘并经过非线性激活函数，然后再与 W_2 相乘。而 Memory Network 中的输入 $\mathbf{x}$ 会与键矩阵 K 相乘计算相似度，并通过 softmax 操作得到注意力分布，最后对值矩阵 V 进行加权求和。

因此，Transformer 中的 FFN 可以视为 Memory Network 的一种形式，其中通过矩阵乘法 W_1 和 W_2 分别模拟了 Memory Network 中的键值矩阵 K 和 V 的作用，非线性激活函数相当于 Memory Network 中的 softmax 操作。

2.4

设 W^O 按列分块为 $[W_1^O, W_2^O, \dots, W_h^O]$ ，其中每块 $W_j^O \in \mathbb{R}^{d_v \times d}$ ，多头注意力结果可分解为：

$$\text{MultiHead}(Q, K, V) = \sum_{j=1}^h \text{head}_j \cdot W_j^O$$

证明： $\text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{n \times h \cdot d_v}$ ；分块乘法等价于各头独立乘对应子矩阵后相加，即：

$$\text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O = \sum_{j=1}^h \text{head}_j \cdot W_j^O$$

5. 忽略 Transformer 架构中的 LayerNorm，用 MHA 表示多头注意力机制，FFN 表示前馈神经网络，给定 Transformer 第 l 层输入 x^l ，那么第 l 层的输出是什么？答案示例： $MHA(x^l) + FFN(x^l)$ 。[提示：不要忘记残差流] (10 points)

解：

第 l 层输出公式：

$$x^{l+1} = x^l + \text{MHA}(x^l) + \text{FFN}(x^l + \text{MHA}(x^l))$$

1. 计算多头注意力并加残差： $h^l = x^l + \text{MHA}(x^l)$
2. 计算 FFN 并加残差： $x^{l+1} = h^l + \text{FFN}(h^l)$

三. (20 points) 神经网络和注意力机制的实现

给定一个批次的序列输入 `input = torch.Tensor(32, 100, 500)`，其中 32 表示批次大小，100 表示序列长度，而 500 表示维度数。

1. 请学习 pytorch 中关于卷积层，LSTM 以及 Transformer 编码器的接口，即 `nn.LSTM()` 等。请依次定义上面三个神经网络让其接受 `input` 为输出，输出一个跟 `input` 同等形状的 tensor。(10 points)
2. 请实现任意两种注意力机制，对该序列输入进行聚合，得到一个新的聚合后的 tensor，该 tensor 的形状为 (32,500)。(10 points)

解：

```
1 import torch
2 import torch.nn as nn
3
4 # 1. 定义三种神经网络层
5 class NeuralLayers(nn.Module):
6     def __init__(self):
7         super().__init__()
8         # 卷积层 (保持序列长度不变)
9         self.conv = nn.Conv1d(
10             in_channels=500, # 输入特征维度
11             out_channels=500, # 输出特征维度
12             kernel_size=3, # 卷积核尺寸
13             padding=1, # 保持时序长度不变
14             padding_mode='replicate'
15         )
16
17         # LSTM层
18         self.lstm = nn.LSTM(
19             input_size=500,
20             hidden_size=500, # 保证输出维度一致
21             batch_first=True,
22             bidirectional=False
23         )
24
25         # Transformer 编码器层
26         encoder_layer = nn.TransformerEncoderLayer(
27             d_model=500,
28             nhead=10, # 500 必须能被 nhead 整除
29             dim_feedforward=2048,
30             batch_first=True
31         )
32         self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=1)
33
34     def forward(self, x):
35         # 卷积层处理 (需要调整维度顺序)
36         conv_out = self.conv(x.permute(0,2,1)).permute(0,2,1)
37
38         # LSTM处理
39         lstm_out, _ = self.lstm(x)
```

```
40
41     # Transformer处理 (需要添加位置编码)
42     transformer_out = self.transformer(x)
43
44     return conv_out, lstm_out, transformer_out
45
46 # 2. 实现两种注意力机制
47 class AttentionMechanisms(nn.Module):
48     def __init__(self, d_model=500):
49         super().__init__()
50         # 可学习参数初始化
51         self.query = nn.Parameter(torch.randn(d_model))
52         self.W = nn.Linear(d_model, d_model)
53
54     # 方法1: 加性注意力
55     def additive_attention(self, x):
56         # x.shape: (32, 100, 500)
57         energy = torch.tanh(self.W(x))          # (32, 100, 500)
58         attn_weights = torch.softmax(
59             energy @ self.query,                 # (32, 100)
60             dim=1
61         )
62         return (attn_weights.unsqueeze(-1) * x).sum(dim=1) # (32, 500)
63
64     # 方法2: 多头缩放点积注意力
65     def scaled_dot_attention(self, x):
66         Q = K = V = x # 自注意力
67         attn_logits = torch.matmul(Q, K.transpose(-2, -1)) # (32, 100, 100)
68         attn_weights = torch.softmax(
69             attn_logits / torch.sqrt(torch.tensor(Q.size(-1))),
70             dim=-1
71         )
72         return torch.matmul(attn_weights, V).mean(dim=1) # (32, 500)
73
74     def forward(self, x):
75         out1 = self.additive_attention(x)
76         out2 = self.scaled_dot_attention(x)
77         return out1, out2
78
79 # 验证代码
80 if __name__ == "__main__":
81     input_tensor = torch.randn(32, 100, 500)
82
83     # 测试第一部分
84     model = NeuralLayers()
85     conv_out, lstm_out, transformer_out = model(input_tensor)
86     print(f"卷积输出形状: {conv_out.shape}") # 应为(32, 100, 500)
87     print(f"LSTM输出形状: {lstm_out.shape}") # 应为(32, 100, 500)
88     print(f"Transformer输出形状: {transformer_out.shape}") # 应为(32, 100, 500)
89
90     # 测试第二部分
91     attn_model = AttentionMechanisms()
```

```
92     attn1, attn2 = attn_model(input_tensor)
93     print(f"加性注意力输出形状: {attn1.shape}") # 应为 (32, 500)
94     print(f"点积注意力输出形状: {attn2.shape}") # 应为 (32, 500)
```

Listing 1: 神经网络和注意力机制的完整实现

运行结果:

```
卷积输出形状: torch.Size([32, 100, 500])
LSTM输出形状: torch.Size([32, 100, 500])
Transformer输出形状: torch.Size([32, 100, 500])
加性注意力输出形状: torch.Size([32, 500])
点积注意力输出形状: torch.Size([32, 500])
```

四. (25 points) 词嵌入

本题探索词嵌入并且用词嵌入进行文本分类。

1. 请解释 Word2vec 中 CBOW 和 Skip-gram 两种模型的区别，并说明各自的适用场景。(5 points)
2. 请基于词嵌入（如 Glove 50d）和机器学习分类器（如 SVM 等）对 AG news 数据集进行文本分类。请提供代码并且给出在测试集上的分类结果。代码应包括加载词嵌入，文本到词嵌入的映射，训练分类器。如果受限于设备，可以使用一部分数据集进行训练。(12 points)
3. 在仍然只使用该词嵌入和分类器的情况下，有没有什么方法可以提升性能并且验证。(8 points)

4.1

区别：

- CBOW：通过上下文词预测中心词。输入层接收窗口内所有上下文词的向量，隐含层对这些向量取平均，输出层通过 Softmax 预测中心词的概率分布
- Skip-gram：通过中心词预测上下文词。输入为中心词的向量，输出层预测窗口内每个上下文词的概率

适用场景：

- CBOW：
 - 大规模语料库：因上下文词的平均化操作能高效处理高频词
 - 实时性要求高的场景：如新闻标题生成、实时翻译
- Skip-gram：
 - 小规模或长尾数据：如学术术语、专业领域词汇的语义建模
 - 需要精细语义分析的任务：如诗歌生成、同义词挖掘

4.2

```
1 import pandas as pd
2 import numpy as np
3 import torch
4 from sklearn.svm import SVC
5 from sklearn.metrics import accuracy_score
6 from tqdm import tqdm
7 import re
8
9 # 1. 加载数据集
10 df = pd.read_csv("/root/DL-lab/lab5/ag/train.csv") # 请替换成你的文件路径
11 df.columns = ["label", "title", "description"] # CSV 有3列：标签，标题，描述
12 df["text"] = df["title"] + " " + df["description"] # 合并标题和描述作为输入文本
13 df["label"] = df["label"] - 1 # AG NEWS 的标签是 1-4，我们转换成 0-3
14 train_texts, train_labels = df["text"].tolist(), df["label"].tolist()
15 number = int(0.3 * len(train_texts))
```



```
16 train_texts, train_labels = train_texts[: number], train_labels[: number]
17
18 df = pd.read_csv("/root/DL-lab/lab5/ag/test.csv") # 请替换成你的文件路径
19 df.columns = ["label", "title", "description"] # CSV 有3列: 标签, 标题, 描述
20 df["text"] = df["title"] + " " + df["description"] # 合并标题和描述作为输入文本
21 df["label"] = df["label"] - 1 # AG NEWS 的标签是 1-4, 我们转换成 0-3
22 test_texts, test_labels = df["text"].tolist(), df["label"].tolist()
23
24 # 2. 加载GloVe词嵌入 (50维)
25 def load_glove(glove_path):
26     embeddings = {}
27     with open(glove_path, 'r', encoding='utf-8') as f:
28         for line in tqdm(f):
29             values = line.split()
30             word = values[0]
31             vector = np.asarray(values[1:], dtype='float32')
32             embeddings[word] = vector
33     return embeddings
34
35 glove = load_glove("/root/DL-lab/lab4/glove.6B.50d.txt") # 需替换为实际路径
36 embed_dim = 50
37
38 # 3. 文本预处理和向量化
39 def text_to_vector(text, embeddings):
40     # 清洗和分词
41     text = re.sub(r'[~a-zA-Z]', ' ', text.lower())
42     words = text.split()
43
44     # 获取词向量
45     vectors = []
46     for word in words:
47         if word in embeddings:
48             vectors.append(embeddings[word])
49
50     # 处理空文本
51     if len(vectors) == 0:
52         return np.zeros(embed_dim)
53
54     # 平均池化
55     return np.mean(vectors, axis=0)
56
57 # 转换为向量表示
58 print("Processing training data...")
59 X_train = np.array([text_to_vector(text, glove) for text in tqdm(train_texts)])
60 y_train = np.array(train_labels)
61
62 print("Processing test data...")
63 X_test = np.array([text_to_vector(text, glove) for text in tqdm(test_texts)])
64 y_test = np.array(test_labels)
65
66 # 4. 训练SVM分类器
67 svm = SVC(kernel='linear', C=1.0)
```

```
68 svm.fit(X_train, y_train)
69
70 # 5. 评估模型
71 y_pred = svm.predict(X_test)
72 acc = accuracy_score(y_test, y_pred)
73 print(f"Test Accuracy: {acc:.4f}")
```

Listing 2: 基于 GloVe 词嵌入和机器学习分类器的 AG News 文本分类实现

运行结果:

```
400000it [00:04, 89961.50it/s]
Processing training data...
100%|██████████| 36000/36000 [00:02<00:00, 13103.33it/s]
Processing test data...
100%|██████████| 7600/7600 [00:00<00:00, 8065.98it/s]
Test Accuracy: 0.8753
```

4.3

1. 数据预处理优化

- 精细化分词与清洗: 使用 NLTK 或 spaCy 进行词干提取、停用词过滤, 减少噪声
- 数据增强: 通过同义词替换 (WordNet) 或回译 (Back Translation) 扩充训练数据

2. 词向量特征增强

- 动态加权池化: (1) TF-IDF 加权平均: 对重要词赋予更高权重; (2) 注意力机制: 通过可学习权重动态聚合关键语义 (如使用 Transformer 编码器)
- 子词嵌入 (Subword): 结合 FastText 处理未登录词

3. 分类器优化

- 模型选择与调参: (1) 线性模型: 使用 XGBoost 或 MLP 替代 SVM, 捕捉复杂特征交互; (2) 超参数搜索: 调整 SVM 核函数 (RBF/线性)、正则化强度 (C 值)
- 集成学习: 结合多个分类器 (如 SVM+ 随机森林) 投票提升鲁棒性

4. 验证与评估

- 交叉验证: 采用 5 折交叉验证, 确保模型泛化性
- 多维度指标: 除准确率外, 计算 F1-score (处理类别不平衡)、混淆矩阵分析 (识别易混淆类别)
- 可视化分析: 通过 t-SNE 降维观察词向量分布, 验证语义聚类效果